



Food remedy Co. API

Deployment Research Report

Table of Contents

Introduction	2
RE031 - App Store & Play Store Submission Requirements and Guidelines.....	2
RE032 - Performance Optimisation for Mobile Application Deployment	2
RE033 - Mobile App Security and Data Protection Standards	3
RE034 - Privacy Compliance and Data Governance.....	4
RE035 - Scalability and Backend Readiness for Production Deployment	4
RE036 - Testing Methodologies for Mobile Application Deployment	5
RE037 - UX Readiness and App Store Standards	5
RE038 - Post-Deployment Monitoring and Continuous Improvement	6
RE039 - Security, Privacy, and User Data Protection in Mobile Health Applications.....	7
RE040 - Estimated Deployment and Operational Costs for Food Remedy API.....	7
Conclusion	8

Introduction

This consolidated report synthesises the findings of nine individual deployment-focused research tickets completed by the Food Remedy API research team during Trimester 1, 2026. Collectively, these reports address the full spectrum of considerations required to take the Food Remedy mobile application from development into a stable, secure, and compliant production environment. Topics covered include app store submission standards, mobile performance optimisation, UX readiness, privacy regulation, security architecture, backend scalability, testing methodologies, and post-deployment monitoring. Each section below represents one research ticket and credits the original contributing team member.

RE031 - App Store & Play Store Submission Requirements and Guidelines

Original report by: Dhriti Jain

Releasing a mobile application to production requires navigating the formal submission requirements of both the Apple App Store and the Google Play Store, each of which maintains its own set of quality, privacy, and compliance standards. For the Apple App Store, developers must be enrolled in the Apple Developer Program and have access to App Store Connect. Apple's review process is predominantly manual, conducted by human reviewers who evaluate an application against criteria encompassing functionality completeness, interface quality, security, and privacy compliance. Applications must not contain placeholder content, broken interactions, or unfinished screens, as these will result in immediate rejection. Apple also mandates privacy labels; disclosures that detail exactly what data the app collects and whether it is linked to individual users or used for tracking.

The Google Play Store operates through a combination of automated and manual review processes and is generally regarded as offering a faster approval pathway. Developers register once through the Google Play Console and must complete a Data Safety section that explains what data is collected, the purpose behind that collection, whether third parties receive it, the encryption measures applied, and whether users can request deletion. Both platforms share common metadata requirements including an app name, detailed descriptions, high-quality screenshots, an app icon, and a publicly accessible privacy policy. For the Food Remedy application specifically, compliance preparation should include a thoroughly drafted privacy policy, justified permission requests for features such as the camera used in barcode scanning, and accurate store listing materials. Apple's review system places a notably heavier emphasis on user experience quality and privacy rigour, while Google Play's framework is comparatively more flexible.

RE032 - Performance Optimisation for Mobile Application Deployment

Original report by: Sehajpreet Singh

Mobile application performance is not merely a technical concern for a product like Food Remedy, where users expect near-instantaneous barcode scanning results and real-time

nutritional feedback during active shopping, performance is a direct driver of user satisfaction and retention. Industry benchmarks suggest that app launch times should fall below two seconds, frame rendering should target 60 frames per second, and no single frame should take longer than 700 milliseconds to avoid the perception of a frozen interface. Google's Android Vitals platform tracks these metrics in production and flags violations as quality issues.

Several optimisation strategies are particularly applicable to the Food Remedy context. Lazy loading allows the application to defer the initialisation of non-essential screens and modules such as detailed recommendation panels or research-based content until the user explicitly navigates to them, reducing startup overhead. Caching recently scanned barcodes and their associated nutritional results eliminates redundant API calls and ensures that repeat interactions feel instantaneous, even under poor network conditions. On the API side, payloads should be trimmed to return only the fields required by the client, and response bodies should be compressed to minimise data transfer times. Image and asset compression further reduces memory consumption and prevents sluggish startup caused by loading large files unnecessarily. For React Native specifically, avoiding unnecessary component re-renders through memoization and efficient state management is essential, as excessive rendering activity blocks the JavaScript thread and produces UI jank. Testing should always be conducted on real physical devices across both Android and iOS, rather than relying solely on emulators, to capture authentic performance characteristics.

RE033 - Mobile App Security and Data Protection Standards

Original report by: Ocean Ocean

Health and nutrition applications occupy a particularly sensitive position in the mobile security landscape because the data they handle dietary habits, allergen profiles, symptom records, and personalised health goals can reveal intimate details about an individual's lifestyle and physical condition. The Food Remedy API must therefore be designed with security embedded as a foundational concern rather than applied retrospectively. Authentication represents the first critical layer of protection. User credentials should never be stored in plain text; server-side storage must employ modern salted hashing algorithms such as Argon2 or bcrypt. For mobile authentication flows, OAuth 2.0 with PKCE is the recommended approach for public clients, ensuring that access tokens remain short-lived and are stored exclusively within platform-provided secure storage, iOS Keychain or Android Keystore, rather than in accessible file locations or shared preferences.

All communication between the mobile client and the backend API must be conducted exclusively over HTTPS with modern TLS, preventing interception on untrusted networks. Sensitive data persisted on the device should be encrypted at rest, and no health-related information should appear in application logs, crash reports, push notification previews, or analytics payloads. Backend endpoints must enforce object-level authorisation on every request, ensuring that a user cannot access or modify another user's records by manipulating identifiers in API calls; a vulnerability OWASP identifies as one of the most prevalent and damaging API security risks. Rate limiting, input validation, and careful error message sanitisation further protect the API surface from brute-force attacks, injection attempts, and inadvertent information disclosure. The OWASP Mobile Application Security Verification

Standard (MASVS) and the OWASP API Security Top 10 provide comprehensive testing checklists that the team should work through before release.

RE034 - Privacy Compliance and Data Governance

Original report by: Tesha Devi

The Food Remedy application's collection of health-adjacent data; including allergen sensitivities, dietary restrictions, and symptom logs; places it within the scope of heightened privacy obligations under several major regulatory frameworks. The European Union's General Data Protection Regulation (GDPR) is particularly pertinent: under Article 9, data that reveals information about a person's health is classified as a special category of personal data, requiring an explicit and unambiguous basis for collection. Non-compliance exposes organisations to fines of up to €20 million or four percent of global annual turnover. Other relevant frameworks include California's CCPA/CPRA, Canada's PIPEDA, and Brazil's LGPD, all of which share foundational principles around transparency, data minimisation, purpose limitation, and user rights.

For consent to be valid, it must be freely given, specific, and actively obtained; pre-ticked checkboxes and passive scrolling do not constitute consent. The application should present users with granular toggles that allow independent control over different categories of data use, such as analytics, marketing communications, and health-related profiling. Withdrawing consent must be as straightforward as granting it. Privacy policies should be structured in a layered format: a concise plain-language summary suitable for mobile reading, complemented by a full legal policy accessible via the website and a dedicated appendix addressing health data specifically. The team should also implement a Health Data Dashboard enabling users to view, correct, and delete individual data entries, and auto-deletion policies should be established so that symptom logs are purged after approximately ninety days. All third-party SDK providers must sign Data Processing Agreements, and a 72-hour breach notification procedure should be documented and rehearsed.

RE035 - Scalability and Backend Readiness for Production Deployment

Original report by: Sanya

A backend system that performs adequately under development conditions may degrade significantly when exposed to the volume and unpredictability of real-world user traffic. For the Food Remedy API, which must handle concurrent barcode lookups, personalised recommendation computations, and user profile queries, backend scalability is a prerequisite for a reliable production experience. Scalability can be approached through two primary strategies: vertical scaling, which involves augmenting the resources of an individual server, and horizontal scaling, which distributes workloads across multiple servers simultaneously. Modern cloud architectures strongly favour horizontal scaling due to its greater flexibility, fault tolerance, and capacity for growth without the hard limits imposed by individual hardware.

Load balancing is a complementary mechanism that distributes incoming requests across available backend servers, preventing any single instance from becoming a bottleneck. Common load balancing strategies include round-robin distribution, least-connections routing, and geographic load balancing, which directs users to the nearest server to minimise latency. Cloud platforms such as AWS, Google Cloud Platform, and Microsoft Azure provide auto-scaling capabilities that dynamically provision and decommission server instances in response to traffic levels, optimising cost efficiency without sacrificing availability. Containerisation using Docker, combined with Kubernetes orchestration, enables services to be deployed, updated, and scaled independently, reducing the risk of cascading failures. Common backend bottlenecks including database overload, API latency, memory leaks, and single points of failure can be mitigated through indexing, caching, distributed architecture, and real-time monitoring tools such as Prometheus and Grafana.

RE036 - Testing Methodologies for Mobile Application Deployment

Original report by: Gurkirat Singh Chahal

Rigorous testing before deployment is the primary mechanism through which development teams can build confidence in an application's stability, security, and usability across the diverse range of devices and operating system versions it will encounter in production. The mobile application testing lifecycle encompasses several distinct phases. Unit testing examines individual functions and components in isolation, enabling developers to catch logic errors and regressions early in the development cycle. Integration testing verifies that modules interact correctly; confirming, for example, that the barcode scanning frontend communicates accurately with the nutritional data backend and that authentication integrates seamlessly with cloud services. Usability testing engages real users to evaluate navigation clarity, accessibility, and interface intuitiveness, producing qualitative feedback that automated processes cannot capture.

Additional testing types essential to deployment readiness include performance testing, which measures speed, responsiveness, and resource consumption under load; compatibility testing, which ensures consistent behaviour across different device manufacturers, screen sizes, and OS versions; and security testing, which identifies vulnerabilities before they can be exploited. The Android and iOS platforms present distinct testing challenges: Android's extensive device fragmentation necessitates broad compatibility testing across numerous hardware configurations and OS versions, while iOS testing benefits from a more standardised hardware ecosystem but demands strict adherence to Apple's Human Interface Guidelines. Tools commonly employed in mobile testing include Appium for cross-platform automation, Firebase Test Lab for cloud-based Android testing across physical and virtual devices, JUnit for Java unit testing, and XCTest for iOS-specific testing. Automated testing reduces human error and accelerates regression detection, while manual usability testing with real participants captures experiential dimensions that automation cannot replicate.

RE037 - UX Readiness and App Store Standards

Original report by: Adithi Rajesh Nair

User experience readiness is a prerequisite for app store approval and long-term user retention alike. Both Apple and Google evaluate applications not only against functional criteria but also against design quality standards that reflect their respective platform conventions. Apple's Human Interface Guidelines emphasise visual consistency, intuitive navigation, and responsiveness across device sizes, while Google's Material Design system provides analogous guidance for Android applications. An application that functions correctly but presents a confusing or aesthetically inconsistent interface risks rejection during review or, if approved, poor user ratings that diminish its discoverability. For Food Remedy, UX readiness encompasses ensuring that the barcode scanning workflow is frictionless and provides immediate visual feedback, that nutritional information is presented in a clear and legible hierarchy, and that allergen warnings are prominently surfaced without requiring additional navigation. Accessibility considerations including adequate contrast ratios, touch target sizing, and support for screen readers are also evaluated during platform review processes and are increasingly important to a broad user base.

RE038 - Post-Deployment Monitoring and Continuous Improvement

Original report by: Prajjwal Makkar

Deployment marks the beginning, not the conclusion, of an application's operational lifecycle. Real-world usage consistently reveals performance characteristics, usage patterns, and failure modes that do not manifest during controlled testing and for a system like Food Remedy, where users depend on accurate recommendations and responsive interactions, undetected issues translate directly into eroded trust. Firebase Analytics is well-suited to tracking behavioural metrics such as daily active users, session duration, feature engagement rates, and retention curves, enabling the team to identify which aspects of the application resonate with users and which are abandoned. Crashlytics complements this by providing real-time crash detection and device-level diagnostic information, allowing developers to prioritise fixes according to the breadth of impact. For backend infrastructure, Prometheus and Grafana provide visibility into server health, API response times, memory utilisation, and database performance through configurable real-time dashboards and alerting rules.

User feedback mechanisms are equally important alongside telemetry data. Application reviews and ratings, in-app feedback forms, support request patterns, and structured beta testing communities collectively illuminate the qualitative dimensions of user experience that metrics alone cannot convey. The distinction between what users do as captured by analytics, and why they do it as revealed through feedback, is essential for informed product decisions. The principle of continuous improvement holds that deployments should be treated as iterative rather than terminal events; smaller, more frequent updates grounded in behavioural data and user sentiment allow the product to evolve responsively rather than accumulating technical debt through infrequent large releases. The Food Remedy team should adopt agile improvement cycles and establish clear ownership of monitoring dashboards to ensure that post-deployment signals translate reliably into product enhancements.

RE039 - Security, Privacy, and User Data Protection in Mobile Health Applications

Original report by: Riken Patel

Mobile health and nutrition platforms handle information that extends well beyond ordinary app data; dietary preferences, allergy profiles, product search histories, and personalised recommendations collectively constitute a detailed portrait of a user's health behaviours and physiological sensitivities. In Australia, health information is classified as sensitive information under the Privacy Act, and the Australian Privacy Principles impose strict obligations on how such data may be collected, stored, and disclosed. The Food Remedy API must therefore ensure that its data handling practices are not merely technically competent but genuinely privacy-preserving by design. Specific risks identified through OWASP's Mobile Top 10 framework include insecure data storage, inadequate privacy controls, insufficient cryptography, and insecure communication; all of which are directly applicable to a food and health tracking context.

Concrete protective measures for the Food Remedy application include enforcing HTTPS across all API communication, storing authentication tokens only within platform-provided secure storage rather than in plain text, validating user permissions on every individual API request to prevent unauthorised data access, and returning the minimum data set necessary to satisfy each frontend request. Allergy and dietary data must be encrypted both in transit and at rest, and debug logs must be sanitised to ensure that sensitive user attributes are never inadvertently captured. Rate limiting on API endpoints protects against automated abuse, while input validation prevents malformed or malicious payloads from reaching the application's data layer. Clear, accessible privacy information and consent mechanisms that allow users to understand and control how their data is used are not optional enhancements as they are foundational to the trust relationship upon which the application's ongoing use depends.

RE040 – Estimated Deployment and Operational Costs for Food Remedy API

Original report by: Prajjwal Makkar

Deploying the Food Remedy application involves both initial publishing costs and long-term operational expenses that continue after release. Before an application can be distributed publicly, developers must register official developer accounts with both Apple and Google. Google Play requires a one-time developer registration fee of approximately USD \$25, while Apple requires enrolment in the Apple Developer Program, which currently costs USD \$99 annually. Beyond these publishing fees, production deployment also introduces infrastructure-related operational costs that increase as the user base grows.

The Food Remedy API will require cloud infrastructure to support barcode scanning requests, personalised recommendation generation, user authentication, analytics collection, and database storage. Platforms such as Firebase, Google Cloud Platform, AWS, and Render provide scalable hosting environments suitable for early-stage mobile deployment. Initially, free-tier services may support low user traffic; however, operational costs gradually increase due to higher API usage, increased database storage, cloud server scaling, monitoring

systems, and analytics processing. Additional third-party services such as Firebase Analytics, Crashlytics, authentication providers, email systems, and AI-based APIs may also contribute recurring monthly costs once usage exceeds free-tier limits.

To maintain cost efficiency, the Food Remedy team should adopt scalable infrastructure strategies rather than investing heavily in enterprise-grade systems during early deployment stages. Backend optimisation practices such as caching, efficient database indexing, compressed API responses, and reduced unnecessary requests can significantly minimise operational expenses. Gradual scaling through cloud-based auto-scaling services allows infrastructure costs to grow proportionally alongside user adoption rather than requiring large upfront investments. By combining affordable cloud infrastructure with scalable deployment practices, the Food Remedy application can maintain production readiness while controlling long-term operational costs effectively.

Conclusion

The combined findings of these deployment-focused research tickets provide a comprehensive foundation for preparing the Food Remedy application for real-world production deployment. Across all sections, several key themes consistently emerged, including the importance of application performance, security, privacy protection, scalability, testing reliability, user experience quality, and continuous monitoring after release. Together, these areas form the core requirements of building a stable, trustworthy, and production-ready mobile health application.

The research also highlighted that successful deployment extends beyond simply publishing an application to the App Store or Play Store. Long-term success depends on maintaining reliable backend infrastructure, protecting sensitive user information, monitoring application behaviour continuously, and responding quickly to technical or usability issues. In addition, deployment planning must also consider operational sustainability, including cloud infrastructure expenses, monitoring systems, third-party services, and scalability-related costs that increase as user adoption grows.

By implementing the strategies and recommendations identified throughout these research tickets, the Food Remedy team can create a deployment environment that is secure, scalable, user-focused, and financially sustainable. Collectively, these findings position the application for a stronger production launch while supporting future growth, continuous improvement, and long-term user trust.